

COMPENG 2SH4 Project – Peer Evaluation [30 Marks]

Your Team Members Andrew Honoris (honorisa) Anushka Chauhan (chauha30)

Team Members Evaluated plenej1 seenivar

Provide your genuine and engineeringly verifiable feedback. Ungrounded claims will lead to deductions. Completing the peer code evaluation on time will earn your team a total of **30 marks**. Do not exceed 2 paragraphs per question.

Peer Code Review: OOD Quality

1. **[3 marks]** Examine the main logic in the main program loop. Can you easily interpret how the objects interact with each other in the program logic through the code? Comment on what you have observed, both positive and negative features.

The main program loop is very object-oriented, with almost every line being some reference to another object. The code in the main program is quite easy and logical to follow, though it is more difficult to follow in the subclasses where the actual logic occurs, without any comments, as will be mentioned later.

2. **[3 marks]** Quickly summarize in point form the pros and cons of the C++ OOD approach in the project versus the C procedural design approach in PPA3.

Pros of C++ Object-Oriented Design

- OOD allows people to work in parallel, leading to theoretically faster coding times
- Splitting up all the logic may make it easier to pinpoint where the issue lies (eg, Food class, Player class, etc.) if you know which part is failing
- Better reusability and easier to extend with more features, which helps with more advanced game features to be easily tested and manipulated
- Internal data are protected from others so that unwanted changes can be avoided with encapsulation

Cons of C++ Object-Oriented Design

- More complex due to rule of 4 or 6 as incorrect use of memory allocation can cause crashes.
- Too many objects can make it harder to visualize.
- Debugging is harder,, if you do not know why your code is failing, can lead to failures which you do not know the origin of. The root cause may be completely unrelated to what is suspected to be the problem, and thus it may take much longer to debug

Pros of C procedural design

- May be more straightforward to develop and read/understand due to its linear development; reads more like a “book”/”list of instructions
- Less code needed in terms of setting up objects/data types as C doesn’t require constructors
- Less run-time support necessary, due to not needing to support more/advanced features that C++ does

Cons of C procedural design

- No native object declaration support, which may lead to more intuitive programming. This leads to the need to work with other data types such as structs
- Harder to expand and add more advanced features. Functions can be easily overwritten unintentionally.
- May be harder to follow where the problem is when everything is written down linearly, as everything is put together

Peer Code Review: Code Quality

1. **[3 marks]** Does the code offer sufficient comments, or deploys sufficient self-documenting coding style, to help you understand the code functionality more efficiently? If any shortcoming is observed, discuss how you would improve it.

There are essentially no comments provided at all, outside of the ones provided by default. As a result, there is not much in terms of self-documentation in this regard, which leads to the code logic being more difficult to follow and confusing at times. With respect to commenting, self-documentation and making the code more readable to other parties, can be promoted through adding comments to specific blocks of logic, explaining variable names, and overall providing a walkthrough of the general logic.

2. **[3 marks]** Does the code follow good indentation, add sensible white spaces, and deploys newline formatting for better readability? If any shortcoming is observed, discuss how you would improve it.

There is evidence of good indentation and white spacing, however newlines are done in an inconsistent manner; sometimes there are 1 or 2 new lines, sometimes there are many more. Sticking to a consistent one new line in between blocks or new “ideas” can lead to distinct blocking, readable code, and much less or no unnecessary spaces.

Peer Code Review: Quick Functional Evaluation

1. **[3 marks]** Does the Snake Game offer smooth, bug-free playing experience? Document any buggy features and use your COMPENG 2SH4 programming knowledge to propose the possible root cause and the potential debugging approaches you’d recommend the other team to deploy. (NOT a debugging report, just technical user feedback)

The snake game experience is smooth and bug-free. While the food generation logic is different from our own as new food gets generated only after the snake has eaten all the food generated earlier on the screen whereas our design a new food got generated every time the snake ate food, this is likely simply a difference in interpretation.

2. **[3 marks]** Does the Snake Game cause memory leak? If yes, provide a digest of the memory profiling report and identify the possible root cause(s) of the memory leakage.

No, the game does not cause any memory leak.

```

~~Dr.M~~      0 unique,      0 total warning(s)
~~Dr.M~~      0 unique,      0 total,      0 byte(s) of leak(s)
~~Dr.M~~      0 unique,      0 total,      0 byte(s) of possible leak(s)
~~Dr.M~~ ERRORS IGNORED:
~~Dr.M~~      22 potential error(s) (suspected false positives)
~~Dr.M~~      (details: C:\Users\andre\Downloads\School Related (Mac)\
Memory-cmd.exe.2888.000\potential_errors.txt)
~~Dr.M~~      4 potential leak(s) (suspected false positives)
~~Dr.M~~      (details: C:\Users\andre\Downloads\School Related (Mac)\
Memory-cmd.exe.2888.000\potential_errors.txt)
~~Dr.M~~      79 unique,    332 total,    39537 byte(s) of still-reachable al
~~Dr.M~~      (re-run with "-show_reachable" for details)
~~Dr.M~~ Details: C:\Users\andre\Downloads\School Related (Mac)\Year 2\DrM
.exe.2888.000\results.txt
You Exited
Your Final Score Is: 0

```

Project Reflection

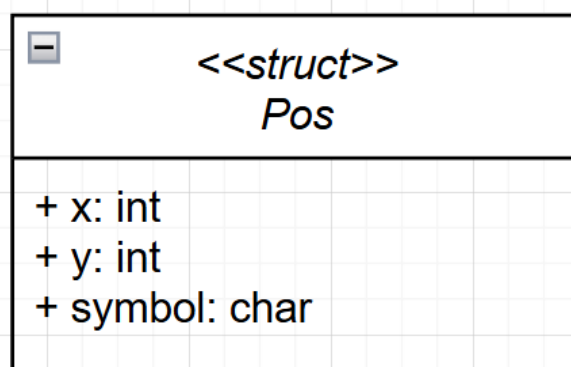
Recall the unusual objPos class design with the additional Pos struct. After reviewing the other team's implementation in addition to yours, reflect on the following questions:

1. **[3 marks]** Do you think the compound object design of objPos class is sensible? Why or why not?

Within the context of a class declaration rather than simply a struct, the objPos class is not sensible. Given the struct itself can hold the values for the coordinates and the symbol, there is no need to have that second layer of having the struct inside of the objPos class, and having to follow the rule of minimum 4, and so on.

2. **[4 marks]** If yes, discuss about an alternative objPos class design that you believe is relatively counterintuitive than the one in this project. If not, explain how you'd improve the object design. You are expected to facilitate the discussion with UML diagram.

To improve the object design, it would be beneficial to look backwards and reference the original Pos struct. There is no need for the Pos to operate with an object interface, rather all the functionality the class provides—getters and setters—can be done through working with simply the struct itself, and does not sacrifice any more or less access than before. This means the (obj)PosArrayList class will instead work with an array of pointers of Pos structs. This again removes the need to satisfy the rule of minimum 4.



Above is a UML diagram of the proposed Pos struct.